

Redis 学习笔记（Java 开发者版）

目标：知道 Redis 是什么、会用常用命令、会用 Java 操作 Redis、理解缓存常见问题和分布式锁。

目录

- 1. Redis 是什么
- 2. 安装与连接
- 3. 常用数据类型与命令
- 4. Key 管理建议
- 5. Java 操作 Redis
- 6. 持久化
- 7. 事务
- 8. 发布订阅
- 9. 缓存常见问题
- 10. 分布式锁
- 11. 高可用与集群
- 12. 内存淘汰策略
- 13. 常用运维命令
- 14. 学习路线建议
- 15. 一句话记忆

1. Redis 是什么

Redis 是一个 **内存 key-value 数据库**，常用来做：

- 缓存
- 计数器
- 排行榜
- 分布式锁
- 消息队列 / 发布订阅
- 会话存储

特点：

- 数据主要放内存，读写非常快
- 支持多种数据结构：String、Hash、List、Set、ZSet 等
- 命令执行主流程是单线程，避免并发竞争
- 默认端口：**6379**
- 默认有 16 个数据库：**0~15**，用 **SELECT index** 切换

Java 类比：

Redis	Java 类似	常见用途
String	Map<String, String>	缓存、计数
Hash	Map<String, Map<String, String>>	存对象

Redis	Java 类似	常见用途
List	<code>LinkedList</code>	队列、栈
Set	<code>HashSet</code>	去重、共同好友
ZSet	<code>TreeSet + score</code>	排行榜

2. 安装与连接

Docker 启动

```
docker run -d --name redis -p 6379:6379 redis:7
docker exec -it redis redis-cli
```

带密码：

```
docker run -d --name redis -p 6379:6379 redis:7 redis-server --requirepass 123456
docker exec -it redis redis-cli -a 123456
```

基本命令

```
PING
SET name "张三"
GET name
DEL name
EXISTS name
EXPIRE name 60
TTL name
PERSIST name
```

3. 常用数据类型与命令

3.1 String

最常用，适合缓存、计数、验证码。

```
SET user:1:name "张三"
GET user:1:name

SETEX code:13800000000 300 "123456"
TTL code:13800000000

INCR article:1:view
DECR article:1:view
```

```
MSET k1 v1 k2 v2
MGET k1 k2
```

3.2 Hash

适合存对象，例如用户信息。

```
HSET user:1 name "张三" age 20
HGET user:1 name
HGETALL user:1
HDEL user:1 age
HINCRBY user:1 age 1
```

Java 类比: `Map<String, Map<String, String>>`。

3.3 List

适合队列、栈、时间线。

```
LPUSH msg:1 "a" "b"
RPUSH msg:1 "c"
LRANGE msg:1 0 -1
LPOP msg:1
RPOP msg:1
LLEN msg:1
```

3.4 Set

适合去重、标签、共同关注。

```
SADD tag:1 java redis
SMEMBERS tag:1
SISMEMBER tag:1 java
SREM tag:1 java
SINTER tag:1 tag:2
```

3.5 ZSet

带分数的有序集合，适合排行榜。

```
ZADD rank 100 tom 90 jerry
ZRANGE rank 0 -1 WITHSCORES
ZREVRANGE rank 0 2 WITHSCORES
```

```
ZSCORE rank tom  
ZINCRBY rank 5 tom
```

4. Key 管理建议

命名规范：

```
业务:对象:id  
user:1:name  
order:1001:status
```

常用命令：

```
EXPIRE user:1 60  
TTL user:1  
PERSIST user:1  
DEL user:1  
UNLINK user:1  
SCAN 0 MATCH user:* COUNT 100
```

注意：

- 生产环境不要用 **KEYS ***，会阻塞 Redis。
- 用 **SCAN** 渐进式遍历。
- 尽量给 key 设置过期时间，避免内存无限增长。

5. Java 操作 Redis

5.1 Jedis

Maven 依赖：

```
<dependency>  
  <groupId>redis.clients</groupId>  
  <artifactId>jedis</artifactId>  
  <version>5.1.0</version>  
</dependency>
```

简单示例：

```
import redis.clients.jedis.Jedis;  
  
public class JedisDemo {  
    public static void main(String[] args) {
```

```
try (Jedis jedis = new Jedis("localhost", 6379)) {
    jedis.auth("123456"); // 没有密码可省略

    jedis.set("name", "张三");
    String name = jedis.get("name");
    System.out.println(name);

    jedis.setex("code:13800000000", 300, "123456");
}
}
```

连接池:

```
import redis.clients.jedis.JedisPool;
import redis.clients.jedis.JedisPoolConfig;

JedisPoolConfig config = new JedisPoolConfig();
config.setMaxTotal(20);
config.setMaxIdle(10);

JedisPool pool = new JedisPool(config, "localhost", 6379);

try (Jedis jedis = pool.getResource()) {
    jedis.set("k", "v");
}
```

5.2 Spring Boot + Spring Data Redis

依赖:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Spring Boot 3.x 配置:

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
      password: 123456
      database: 0
```

Spring Boot 2.x 配置:

```
spring:
  redis:
    host: localhost
    port: 6379
    password: 123456
    database: 0
```

使用 StringRedisTemplate:

```
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

import java.time.Duration;

@Service
public class RedisService {

    private final StringRedisTemplate redisTemplate;

    public RedisService(StringRedisTemplate redisTemplate) {
        this.redisTemplate = redisTemplate;
    }

    public void set(String key, String value) {
        redisTemplate.opsForValue().set(key, value);
    }

    public String get(String key) {
        return redisTemplate.opsForValue().get(key);
    }

    public void setWithExpire(String key, String value, long seconds) {
        redisTemplate.opsForValue().set(key, value, Duration.ofSeconds(seconds));
    }
}
```

常用操作:

```
// String
redisTemplate.opsForValue().set("name", "张三");

// Hash
redisTemplate.opsForHash().put("user:1", "name", "张三");

// List
redisTemplate.opsForList().leftPush("queue", "task1");
```

```
// Set
redisTemplate.opsForSet().add("tag:1", "java", "redis");

// ZSet
redisTemplate.opsForZSet().add("rank", "tom", 100);
```

如果要存 Java 对象，通常配置 JSON 序列化：

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@Configuration
public class RedisConfig {

    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory
factory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);

        StringRedisSerializer keySerializer = new StringRedisSerializer();
        GenericJackson2JsonRedisSerializer valueSerializer =
            new GenericJackson2JsonRedisSerializer();

        template.setKeySerializer(keySerializer);
        template.setHashKeySerializer(keySerializer);
        template.setValueSerializer(valueSerializer);
        template.setHashValueSerializer(valueSerializer);
        template.afterPropertiesSet();
        return template;
    }
}
```

6. 持久化

Redis 是内存数据库，但可以持久化到磁盘。

RDB

- 定时快照
- 文件小，恢复快
- 可能丢失最后一次快照后的数据

```
SAVE
BGSAVE
```

AOF

- 追加写命令日志
- 数据更安全
- 文件更大，恢复慢

配置：

```
appendonly yes
appendfsync everysec
```

混合持久化

Redis 4.0+ 支持 RDB + AOF 混合，兼顾恢复速度和安全性。

7. 事务

Redis 事务不是关系型数据库那种完整事务，不支持回滚。

```
MULTI
SET k1 v1
INCR k2
EXEC
```

取消：

```
DISCARD
```

乐观锁：

```
WATCH k1
MULTI
SET k1 v2
EXEC
```

如果 **k1** 被其他客户端改过，**EXEC** 会失败。

8. 发布订阅

```
SUBSCRIBE channel:1  
PUBLISH channel:1 "hello"
```

特点：

- 简单
- 消息不持久
- 适合实时通知，不适合可靠消息队列

9. 缓存常见问题

9.1 缓存穿透

查询一个不存在的数据，每次都打到数据库。

解决：

- 缓存空值，并设置较短过期时间
- 布隆过滤器
- 参数校验

9.2 缓存击穿

某个热点 key 过期，大量请求同时打到数据库。

解决：

- 互斥锁，只让一个线程重建缓存
- 热点 key 逻辑过期，不直接过期
- 预热

9.3 缓存雪崩

大量 key 同时过期，或 Redis 宕机，数据库压力暴涨。

解决：

- 过期时间加随机值
- Redis 集群 / 哨兵
- 限流、降级
- 多级缓存

9.4 缓存与数据库一致性

常用方案：

1. 先更新数据库，再删除缓存
2. 延迟双删
3. 订阅 MySQL binlog，通过 Canal 等同步删除缓存
4. 缓存设置过期时间作为兜底

没有绝对完美方案，根据业务容忍度选择。

10. 分布式锁

10.1 原生 Redis 思路

加锁：

```
SET lock:order:1001 uuid NX EX 30
```

释放锁要用 Lua 保证原子性：

```
if redis.call('get', KEYS[1]) == ARGV[1] then
    return redis.call('del', KEYS[1])
else
    return 0
end
```

要点：

- value 用唯一值，例如 UUID
- 必须设置过期时间
- 释放锁要判断是不是自己的锁
- 业务没执行完要考虑锁续期

10.2 Redisson

生产环境更推荐 Redisson。

依赖：

```
<dependency>
  <groupId>org.redisson</groupId>
  <artifactId>redisson-spring-boot-starter</artifactId>
  <version>3.27.0</version>
</dependency>
```

示例：

```
import org.redisson.api.RLock;
import org.redisson.api.RedissonClient;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.concurrent.TimeUnit;
```

```
@Service
public class OrderService {

    @Autowired
    private RedissonClient redissonClient;

    public void doBusiness(String orderId) {
        RLock lock = redissonClient.getLock("lock:order:" + orderId);

        try {
            boolean locked = lock.tryLock(10, 30, TimeUnit.SECONDS);
            if (!locked) {
                return;
            }

            // 执行业务
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            if (lock.isHeldByCurrentThread()) {
                lock.unlock();
            }
        }
    }
}
```

11. 高可用与集群

主从复制

- 主节点写
- 从节点读
- 从节点通过 `replicaof` 复制主节点数据

```
replicaof master-host 6379
```

哨兵 Sentinel

- 监控主从
- 主节点挂了自动选新主
- 适合高可用，但不分片

Cluster 集群

- 数据分片
- 共 16384 个 slot
- 通常至少 3 主 3 从
- 适合大数据量、高并发

12. 内存淘汰策略

配置：

```
maxmemory 2gb
maxmemory-policy allkeys-lru
```

常见策略：

- **noeviction**：不淘汰，写入报错
- **allkeys-lru**：所有 key 中淘汰最近最少使用
- **allkeys-lfu**：所有 key 中淘汰最不经常使用
- **volatile-lru**：只淘汰设置了过期时间的 key
- **volatile-ttl**：优先淘汰快过期的 key
- **allkeys-random**：随机淘汰

缓存场景常用：**allkeys-lru**。

13. 常用运维命令

```
INFO
MONITOR
SLOWLOG GET
DBSIZE
FLUSHDB
FLUSHALL
```

注意：

- **MONITOR** 会打印所有命令，生产慎用。
- **FLUSHALL** 会清空所有数据库，极其危险。

14. 学习路线建议

1. 用 Docker 启动 Redis，会 **redis-cli**
2. 练熟 5 种基础类型：String、Hash、List、Set、ZSet
3. 用 Jedis 写一个简单缓存
4. 用 Spring Boot + **StringRedisTemplate** 操作 Redis
5. 理解缓存穿透、击穿、雪崩
6. 用 Redisson 实现分布式锁
7. 了解 RDB、AOF、主从、哨兵、Cluster
8. 最后再看源码、Lua、Stream、布隆过滤器

15. 一句话记忆

- String：缓存、计数
- Hash：对象

- List：队列
- Set：去重
- ZSet：排行榜
- 缓存三坑：穿透、击穿、雪崩
- 分布式锁：SET NX EX + Lua，生产用 Redisson